

COLEGIO VALLE DEL SABER
ASIGNACIÓN: Programación
DOCENTE: Jeffrey Barillas



Cámara de detección en tiempo real

Documentación por: Jonathan Valenzuela
Código hecho por: Santiago Quiñonez
Grupo: Dylan Javier, Jonathan Valenzuela, Alexandre Orozco, Santiago Quiñonez

25/05/2026

Documentacion

```
1     import cv2
2     import mediapipe as mp
3
```

Primero, iniciamos importando la librería OpenCV, la cual cumple la función de captura de video de cámara web y procesamiento de imágenes, y prácticamente se encarga de todas las funciones de video, y la librería Mediapipe, que cumple la función de detección de manos, rostro y puntos clave.

```
# 1. Inicializar MediaPipe Holistic y las herramientas de dibujo
mp_holistic = mp.solutions.holistic
mp_drawing = mp.solutions.drawing_utils
```

En esta parte inicializamos con Mediapipe Holistic, que es el encargado de detectar diferentes partes del cuerpo como el rostro, las manos y la postura. Se crea la variable `mp_holistic` para acceder a las funciones del modelo integral de MediaPipe, y también se define `mp_drawing`, que contiene las herramientas necesarias para dibujar los puntos y conexiones que el modelo detecta en la imagen o video.

```
# Configuración del modelo integral (Rostro + Manos + Postura)
holistic = mp_holistic.Holistic(
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)
```

Esta sección configura la sección de Holistic de Mediapipe que combina la detección de manos, rostro en un solo sistema creando la variable `holistic` para iniciar el modelo con dos

parámetros importantes: `min_detection_confidence` que define el nivel mínimo para la detección correcta de las partes del cuerpo y `min_tracking_confidence` que controla la precisión de seguimiento de los puntos ya detectados.

```
# 2. Función auxiliar para calcular y dibujar el cuadro delimitador (cuadrado)
def dibujar_cuadro(frame, landmarks, color, etiqueta):
    h, w, _ = frame.shape
    x_min, y_min = w, h
    x_max, y_max = 0, 0
```

En esta parte utilizamos un `def` que define la función de `dibujar_cuadro`, que sirve como una herramienta auxiliar para calcular y detectar el cuadro que rodea la parte del cuerpo detectada. La función `frame` recibe el fotograma actual, `landmarks` son los puntos detectados, el color del cuadro y la etiqueta de texto. Luego primero obtiene el tamaño del fotograma en alto y ancho para poder ubicar los puntos correctamente. Luego inicializa las variables que guardarán las coordenadas mínimas y máximas en los ejes X y Y, las cuales se usarán más adelante para determinar los límites del rectángulo que encierra la zona detectada.

```
# Encontrar los puntos extremos de la parte del cuerpo
for lm in landmarks.landmark:
    cx, cy = int(lm.x * w), int(lm.y * h)
    if cx < x_min: x_min = cx
    if cx > x_max: x_max = cx
    if cy < y_min: y_min = cy
    if cy > y_max: y_max = cy
```

En esta sección utilizamos un bucle `for` para recorrer todos los puntos dentro de los `landmarks` que representan las coordenadas de la parte del cuerpo a analizar. El bucle sirve para ir analizando punto por punto calculando su posición multiplicando píxeles según lo requerido a lo largo y ancho del fotograma. Gracias a esto el programa compara los valores actuales mínimos con los máximos guardados actualizándolos cuando encuentra un punto

más extremo y gracias a esto se pueden determinar los límites exactos de los ejes X y Y permitiendo definir el área total que ocupará la parte de cuerpo usada y así dibujando los límites del cuadro que la encierra.

```
# Añadir un pequeño margen al cuadro
margen = 15
x_min = max(0, x_min - margen)
y_min = max(0, y_min - margen)
x_max = min(w, x_max + margen)
y_max = min(h, y_max + margen)
```

En esta sección se añade un pequeño margen al cuadro delimitador para evitar que quede demasiado ajustado a los puntos detectados. Se define la variable `margen` con un valor de 15 píxeles, que se usa para expandir ligeramente el área del rectángulo. Luego, se ajustan las coordenadas mínimas y máximas del cuadro utilizando las funciones `min()` y `max()` para asegurarse de que el margen no se salga de los límites de la imagen.

```
# Dibujar el rectángulo y el texto de la etiqueta
cv2.rectangle(frame, (x_min, y_min), (x_max, y_max), color, 2)
cv2.putText(frame, etiqueta, (x_min, y_min - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
```

En esta parte se realiza el dibujo visual del cuadro y su etiqueta sobre el fotograma. La función `cv2.rectangle()` de OpenCV se encarga de trazar el rectángulo utilizando las coordenadas mínimas y máximas calculadas previamente, el color definido y un grosor de línea de 2 píxeles. Justo después, se usa `cv2.putText()` para escribir la etiqueta correspondiente encima del cuadro, con una fuente legible y el mismo color del rectángulo. En conjunto, estas dos instrucciones permiten mostrar de manera clara y visual la zona detectada dentro de la imagen, facilitando la identificación de cada parte del cuerpo en tiempo real.

```
# 3. Encender la cámara
cap = cv2.VideoCapture(0)

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.flip(frame, 1) # Efecto espejo
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

En esta sección se activa la cámara del dispositivo utilizando la función `cv2.VideoCapture(0)`, donde el parámetro 0 indica que se usará la cámara principal. Luego, se inicia un bucle `while` que se mantiene activo mientras la cámara esté abierta. Dentro del bucle, se leen los fotogramas uno por uno con `cap.read()`, y si en algún momento no se obtiene una imagen válida, el programa se detiene con `break`. Después, cada fotograma se voltea horizontalmente con `cv2.flip(frame, 1)` para crear un efecto espejo, haciendo que los movimientos se vean naturales en pantalla. Finalmente, se convierte el formato de color de BGR a RGB mediante `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)`, ya que MediaPipe trabaja con imágenes en formato RGB. En conjunto, esta parte prepara la captura de video en tiempo real para ser procesada por el modelo de detección.

```
# Procesar todo el cuerpo
resultados = holistic.process(frame_rgb)

# --- DETECCIÓN Y CUADRO DEL ROSTRO ---
if resultados.face_landmarks:
    # Dibujar malla facial opcional (puedes comentarla si solo quieres el cuadro)
    mp_drawing.draw_landmarks(
        frame, resultados.face_landmarks, mp_holistic.FACEMESH_CONTOURS,
        landmark_drawing_spec=None,
        connection_drawing_spec=mp_drawing.DrawingSpec(color=(255, 255, 0), thickness=1)
    )
    # Dibujar cuadro verde para el Rostro
    dibujar_cuadro(frame, resultados.face_landmarks, (0, 255, 0), "Rostro")
```

En esta sección se realiza el procesamiento del fotograma capturado por la cámara mediante el modelo Holistic de MediaPipe. La función `holistic.process(frame_rgb)` analiza la

imagen en formato RGB y detecta las diferentes partes del cuerpo, incluyendo el rostro, las manos y la postura. Luego este verifica si el modelo ha detectado puntos faciales con `if resultado.face_landmarks`. Si existen, se dibuja una malla facial opcional usando `mp.drawing.draw_landmarks`, que conecta los puntos del rostro para mostrar su estructura. Esta malla se puede comentar si solo se desea mostrar el cuadro delimitador. Finalmente, se llama a la función `dibujar_cuadro` para dibujar un rectángulo verde alrededor del rostro detectado y colocar la etiqueta “Rostro”. En conjunto, esta parte del código se encarga de identificar y visualizar el rostro en tiempo real dentro del flujo de video.

```
# --- DETECCIÓN Y CUADRO DE MANO IZQUIERDA ---  
if resultados.left_hand_landmarks:  
    mp_drawing.draw_landmarks(frame, resultados.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS)  
    # Dibujar cuadro cian para mano izquierda  
    dibujar_cuadro(frame, resultados.left_hand_landmarks, (255, 255, 0), "Mano Derecha")
```

En esta sección realiza la detección de la mano izquierda utilizando el modelo de `holistic` de `mediapipe`. Este código verifica si existen puntos detectados para la mano izquierda mediante `if resultados.left_hand_landmarks`. Si se encuentran, se dibujan las conexiones entre los puntos con la función `mp_drawing.draw_landmarks()`, que muestra la estructura de la mano en el fotograma. Después, se llama a la función `dibujar_cuadro()` para crear un rectángulo de color cian alrededor de la mano detectada y colocar la etiqueta que indica.

```
# --- DETECCIÓN Y CUADRO DE MANO DERECHA ---  
if resultados.right_hand_landmarks:  
    mp_drawing.draw_landmarks(frame, resultados.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS)  
    # Dibujar cuadro azul para mano derecha  
    dibujar_cuadro(frame, resultados.right_hand_landmarks, (255, 0, 0), "Mano Izquierda")
```

Esta sección realiza la detección de la mano derecha usando el modelo `Holistic` de `MediaPipe`. El código verifica si existen puntos detectados para la mano derecha mediante `if resultados.right_hand_landmarks`. Si se encuentran, se dibujan las conexiones entre los puntos con la función `mp.drawing.draw_landmarks`, que muestra la estructura completa de la mano en el fotograma. Luego, se llama a la función `dibujar_cuadro()` para crear un rectángulo azul alrededor de la mano detectada y colocar la etiqueta que indica.

```
75
76     # Mostrar la ventana
77     cv2.imshow('Deteccion Multiple con Cuadros', frame)
78
79     if cv2.waitKey(1) & 0xFF == ord('q'):
80         break
81
82     cap.release()
83     holistic.close()
84     cv2.destroyAllWindows()
```

En esta última sección se muestra la ventana principal donde se visualiza la detección en tiempo real mediante la función `cv2.imshow`, que despliega el fotograma procesado con el título “Detección Múltiple con Cuadros”. Luego, el programa espera la pulsación de una tecla con `cv2.waitKey(1)` y, si el usuario presiona la tecla `q`, el bucle se detiene gracias al comando `break`. Al salir del ciclo, se liberan los recursos. `cap.release()` que cierra la cámara, `holistic.close()` que detiene el modelo de MediaPipe y el `cv2.destroyAllWindows()` que cierra todas las ventanas abiertas por OpenCV. Esta parte garantiza el cierre correcto del programa permitiendo liberar memoria y los dispositivos usados en la ejecución.